Notes on ZaP

Wolfgang Grieskamp*

July 26, 2000 – for version 2 of ZaP (DRAFT)

Abstract

This (incomplete and draft) document is an attempt towards a documentation for ZaP, the compiler for Executable Z, as it is part of the ZeTa system.

Contents

1 Introduction **2**

2 Principles **2**

2.1 Recursive Sets . . . 2
2.2 Encapsulated Search . . . 3
2.3 Higher-Orderness . . . 4
2.4 Quantors . . . 4
2.5 Schema Calculus . . . 4
2.6 Declarations . . . 5
2.7 Definition Forms . . . 5
2.8 Free Types . . . 5
2.9 Genericity . . . 6
2.10 Numbers . . . 6
2.11 Power-Sets and the Like . . . 7

3 The User Interface **7**

3.1 Queries . . . 7
3.2 Profiling . . . 8

4 Controlling Search **8**

4.1 Sequentialization . . . 9
4.2 Extensionalization . . . 9

5 Special Definitions **10**

5.1 Debug Printing . . . 10
5.2 Forcing . . . 10
5.3 Assumptions . . . 10

*Technische Universität Berlin, Fachbereich Informatik, Franklinstr. 28/29, 10587 Berlin, e-mail: wg@cs.tu-berlin.de

5.4 Extensionalization 11
5.5 Reduction Over Sets 11
5.6 Reduction Over Sequences 11
5.7 Extensional Powerset 11
5.8 Reflection 11
5.9 Data Input 12

1 Introduction

ZAP (version 2) realizes an execution model for Z based on concepts described e.g. in [1, 2]. It provides a combination of higher-order, functional logic execution principles, adapted to the set-based environment of Z. All idioms of Z which are related to *functional* and to *logic* programming languages are executable with ZAP.

This document is a (first) attempt towards a documentation of ZAP. It cannot substitute a manual and tutorial. For some examples and tutorial introduction, see also the `examples` directory of the ZETA distribution, and the publications [4, 3].

> *Remark*: In order to replay the examples in this document, you may load its source into ZETA[1]. Environments named `zexecexpr` and `zexecpred` contain the execution queries. The source of this document is found in the examples directory of the ZETA distribution, subdirectory `zapnotes`.

2 Principles

2.1 Recursive Sets

As sets are paradigmatic for the specification level of Z, they are for the execution level. Set objects – relations or functions – are eventually defined by (recursive) equations, as in the following example, where we define natural numbers as a free type[2], addition on these numbers and an order relation:

section *Nat*

$N ::= Z \mid S\langle\!\langle \{x : N\} \rangle\!\rangle$

$\mid three == S(S(S\ Z))$

$\mid add : N \times N \longrightarrow N$

$\mid add = \{y : N \bullet (Z, y) \mapsto y\} \cup \{x, y, z : N \mid (x, y) \mapsto z \in add \bullet (S\ x, y) \mapsto S\ z\}$

relation $(_less_)$

$\mid _less_ == \{x, y : N \mid (\exists\, t : N \bullet (x, S\ t) \mapsto y \in add)\}$

We may now execute under ZETA queries such as the following, where we ask for the numbers less then three:

$\{x : N \mid x\ less\ three\}$

$\Rrightarrow$ `{Z,S(Z),S(S(Z))}`

These capabilities are similar to logic programming. In fact, we can give a translation from any clause-based system to a system of recursive set-equations in the style given for *add*, where we collect all clauses for the same relational symbol into a union of set-comprehensions, and map literals $R(e_1, \ldots, e_n)$ to membership tests $(e_1, \ldots, e_n) \in R$.

[1] zeta

[2] See section 2.8 why we use $S\langle\!\langle \{x : N\} \rangle\!\rangle$ instead of $S\langle\!\langle N \rangle\!\rangle$.

2.2 Encapsulated Search

A binary relation R can be *applied*, written as $R\,e$, which is syntactic sugar for the expression $\mu\, t : X \mid (e, t) \in R$. This expression is defined iff their exists a unique t such that the constraint is satisfied; it then delivers this t. The set *add* is a binary relation (since it is member of the set $\mathbb{P}((N \times N) \times N)$), and therefore we can for example evaluate

$add(three, three)$

```
⇛ S(S(S(S(S(S(Z))))))
```

Note the semantic difference of $(e, y) \in R$ and $y = R\,e$: the first is not satisfied if R is not defined at e, or produces several solutions for y if R is not unique at e, whereas the second is *undefined* in these cases.

This difference is resembled in the implementation: application, μ-expressions, and related forms are realized by *encapsulated search*. When resolving the constraint $y = \mu\, t : X \mid (e, t) \in R$ (the unsugared form of $y = R\,e$), the constraint $(e, t) \in R$ is resolved in a different search space then the outer constraint $y = \ldots$ (which just waits for the completion of the inner search).

Encapsulated search is not allowed to produce bindings for variables belonging to outer contexts. If an encapsulated search requires the binding of a context variable, it residuates until the context provides the binding. One reason for this behavior is that we may use the context variables in negative positions (i.e. $\mu\, t : X \bullet \neg\, (x, t) \in S$, where a binding for x obviously is a semantic error). Another reason is that encapsulated search gives us a tool to control resolution order in a natural way – it introduces *functional* reduction into our framework.

As a consequence, in our above example, $y = \mu\, t : X \mid (e, t) \in R$, if we have $e = x$, where x is a variable from an outer context (i.e. if we have originally written $y = R\,x$), the encapsulated search residuates until x is bound in the context – if the resolution of $(x, t) \in R$ actually requires a binding for x [3].

For illustration, we redefine the *less* relation from the previous section as follows:

relation ($_less_1_$)

$\mid _less_1_ == \{x, y : N \mid (\exists\, t : N \bullet y = add(x, S\, t))\}$

$\{x : N \mid x\ less_1\ three\}$

```
⇛  unresolvable constraints in toplevel goal {
      resolving LTX:zapnotes.tex(136.21-136.64)
      with variables:
       x = _x
       y = S(S(...))
       t~1 unbound
      unresolved constraints:
       LTX:zapnotes.tex(136.52-136.61) waiting for subgoal {
         resolving '{_y|(_x,_y) IN _S}'
         with parameters:
          _S = LTX:zapnotes.tex(70.6-70.33)+LTX:zapnotes.tex(71.6-71.67)
          _x = (_x,S(...))
         with variables:
          _y unbound
         unresolved constraints:
          '(_x,_y) IN _S' waiting for variable _x
       }
    }
  {*unknown*}
```

[3] A situation where the value of x is not required is f.i. given if $R = \{x : \mathbb{Z} \bullet (x, x)\}$.

Here, the encapsulated search for $add(x, y)$ cannot continue, since it is not allowed to produce bindings for the context variables x. In the above diagnostics, `{_y|(_x,_y) IN _S}` is the encapsulated search goal for application, which is parameterized over `_x` and `S`. The parameter names are in a different name scope, hence the parameter binding `_x = (_x,S(...))` is *not* cyclic. The constraint residuates for the variable `_x`, which is the value assigned to `x`.

2.3 Higher-Orderness

As functions are first-order citizens in functional languages, sets are in ZAP. For example, we define a function describing relational image as follows:

$$[X, Y]$$
$$image == \lambda R : \mathbb{P}(X \times Y);\ S : \mathbb{P}\, X \bullet \{x : X;\ y : Y \mid x \in S;\ (x, y) \in R \bullet y\}$$

A query for the relational image of the *add* function over the cartesian product of the numbers less then three yields in:

let $ns == \{x : N \mid x\ less\ three\} \bullet image(add, ns \times ns)$

⇛ `{Z,S(Z),S(S(Z)),S(S(S(Z))),S(S(S(S(Z))))}`

In a similar style as above most constants of the Z toolkit (domain restriction and so on) can be naturally defined.

2.4 Quantors

Universal quantification is executable if it deals with finite ranges. For example, we can define a generic constant denoting the set of partial functions:

$$[X, Y]$$
$$pfun == \{R : \mathbb{P}(X \times Y) \mid (\forall x : X \mid x \in \mathsf{dom}\, R \bullet \exists_1 y : Y \bullet (x, y) \in R)\}$$

Universal and unique existential quantification are resolved by enumeration (an instance of encapsulated search). Thus, if we try to check whether *add* is a partial function, we get:

$add \in pfun[N \times N, N]$

⇛
```
...
unresolved constraints:
   LTX:zapnotes.tex(205.58-205.68) waiting for variable y
```

The reason for this behavior is that the domain of *add* is not finitely enumerable:

$\mathsf{dom}\ add$

⇛ `{(Z,y),(S(Z),z),(S(S(Z)),y),(S(S(S(Z))),y),...}`

However, if we restrict *add* to a finite domain it works:

$\exists_1 ns == \{x : N \mid x\ less\ three\} \bullet ((ns \times ns) \lhd add) \in pfun[N \times N, N]$

⇛ `*true*`

Above, $A \lhd R$ restricts the domain of R to the set A; the existential quantor is used to introduce a local name in the predicate.

2.5 Schema Calculus

In Z, a schema denotes a set of *bindings*, where bindings are tuples (records) with named components. The schema calculus operators, then, are essentially set operations with special implicit

signature translations.

Therefore, the schema calculus is fully executable. For instance

$[x, y : \mathbb{Z} \mid y \in 1 \mathinner{..} 2] \land [y, z : \mathbb{Z} \mid y = z]$

$\Rightarrow$ `{<x==x,y==1,z==1>,<x==x,y==2,z==2>}`

For more examples see the birthday-book specification in the `examples` directory of the ZETA distribution.

2.6 Declarations

In Z specifications we often find declarations of the kind $f : A \pfun B$, imposing that f has type $\mathbb{P}(A \times B)$ on the one hand, and that $f \in A \pfun B$ on the other. As we have seen in section 2.4, the check whether a value is a partial function is only executable for finite domains.

In order to conveniently support this specification style, ZAP treats declarations $x : E$ as *assumptions*. Hence the membership test $x \in E$ implied by $x : E$ is discarded for execution.

If $x : E$ is actually a non-redundant property, it has to be explicitely denoted in the constraint part of schema text. For this reason, in the definition of *pfun* in the previous section, we have written $\forall\, x : X \mid x \in \mathsf{dom}\, R \bullet \ldots$ instead of $\forall\, x : \mathsf{dom}\, R \bullet \ldots$.

2.7 Definition Forms

When providing definitions of relations and functions one has to use equational forms. For example, the factorial function would be typically specified in Z by axioms of the kind:

section *facNonExec*

$$\begin{array}{|l} fac : \mathbb{N} \longrightarrow \mathbb{N} \\ \hline fac\, 0 = 1;\ \forall\, x : \mathbb{N} \mid x > 0 \bullet fac(x) = x * fac(x-1) \end{array}$$

A definition of this form is unamenable to execution. Instead one may write:

section *facExec*1

$$\begin{array}{|l} fac : \mathbb{N} \longrightarrow \mathbb{N} \\ \hline fac = \{(0,1)\} \cup (\lambda\, x : \mathbb{N} \mid x > 0 \bullet x * fac(x-1)) \end{array}$$

or

section *facExec*2

$$\begin{array}{|l} fac : \mathbb{N} \longrightarrow \mathbb{N} \\ \hline fac = \lambda\, x : \mathbb{N} \bullet \textbf{if}\ x = 0\ \textbf{then}\ 1\ \textbf{else}\ x * fac(x-1) \end{array}$$

The automatic conversion of axioms of the first form to definitions of the later forms may become a feature of future versions of ZAP[4].

2.8 Free Types

Generators Free types are translated by ZAP to sets which generate the type's domain. Consider the following definition:

section *FreeTypes* $\qquad MySeq ::= nil \mid cons\langle\!\langle 1 \mathinner{..} 3 \times MySeq \rangle\!\rangle$

We can execute:

MySeq

[4] ZAP version 1 provided such conversions, which are, however, not yet ported.

⇛ `{nil,cons(1,nil),cons(1,cons(1,nil)),...}`

Using Constructors Constructors are functions as other functions are, which have the property that they are *(partial) injections*. Partiality may be present because of the restrictions given for the domain in a free type declaration.

We can use $cons^{\sim}$ to convert a constructor to a projection, and $x \in \mathsf{ran}\, cons$ to test whether a value is a variant of this constructor.

Regarding the "matching" of constructed values. in general the constraint $ys' = cons(x, ys)$ can not be used if x and ys are free. The call to *cons* will residuate until x and ys are bound, since the encapsulated search implementing the call tries to execute the constraint $(x, ys) \in \mathsf{dom}\, cons$, to which end it is not allowed to produce bindings for x and ys (cf. section 2.2). Thus, for matching one has to generally write $(x, ys) \mapsto ys' \in cons$.

However, the constraints on the domain of a constructor can be completely discarded, using the following declaration form (cf. section 2.6):

section *FreeTypes*2 $\quad MySeq ::= nil \mid cons\langle\!\langle \{x : 1 \,..\, 3 \times MySeq\} \rangle\!\rangle$

Now it becomes possible to write $ys' = cons(x, ys)$ for matching, since there are no constraints imposed on x and ys which need to be resolved in the encapsulated search of the constructor application. Note, however, that now the assertion that the first component of *cons* is in the range $1 \,..\, 3$ is not longer checked for a constructor call at runtime.

Note that the above treatment is even necessary if we have total constructor functions, e.g. $cons\langle\!\langle \mathbb{Z} \times MySeq \rangle\!\rangle$: the reason is that $ys \in MySeq$ is a constraint which though semantically is true, acts as a generator for values of *MySeq*.

2.9 Genericity

The genericity concept of Z allows to provide instantiations with particular sets. For instance, given the definition of the identity relation:

section *Genericity*

$$[X]$$
$$id == \{x : X \mid x \in X \bullet (x, x)\}$$

. . . we can provide instantiations as follows:

$id[\{1, 2\}]$

⇛ `{(1,1),(2,2)}`

On the implementation level this feature requires to represent each generic constant as a function over its instantiation. Since in most cases the instantiation is universal and imposes no constraints this is a serious efficiency problem.

For this reason, ZAP compiles for each generic constant two versions: one as a function over the instantiation argument, and the other as a constant optimized for the case that the instantiation is universal. Which of these versions is selected is controlled by whether an instantiation of a generic constant is *inferred* or *provided*: for inferred instantiations the optimized version is used.

2.10 Numbers

Though it is possible in our model to represent e.g. natural numbers by constructors Z and $S\langle\!\langle N \rangle\!\rangle$, as shown in the previous sections, enabling resolution techniques on them, this approach is hopelessly inefficient for real applications. Thus numbers are actually integrated by a native

implementation, and resolution techniques for arithmetic constraints are not available for them [5]

To generate extensional number ranges, $n \,..\, m$ may be used:

$\{x : \mathbb{Z} \mid x \in 1 \,..\, 1024;\ x \bmod 2 = 0\}$

$\Rightarrow$ `{2,4,6,8,10,12,14,16,18,20,22,24,26,28,30,32,34,36,38,40,42,44,46,48,50,52, 54,56,58,60,...}`

Note that $n \,..\, m$ forces the storage of a set extension in memory. Thus n and m should be in a moderate range. Use $\{x : \mathbb{Z} \mid n \leq x \leq m\}$ if you do not actually want to enumerate a range.

2.11 Power-Sets and the Like

The resolution techniques for subset constraints in our model are relatively weak (they capture just functional and logic computation, not more). For example, we are not able to enumerate the set $\{x : \mathbb{P}\, A \mid x \subseteq S\}$ (which equals $\mathbb{P}\, S$), even if A is finitely enumeratable (indeed, we can test whether a ground value is in this set).

Technically, the constraint resolution system of ZAP has problems with the following kind of constraints:

- subset constraints where the left or the right hand side is a free variable: $x \subseteq v$ or $v \subseteq x$. These constraints can generally not be resolved before x becomes bound. A typical instance is the powerset example given above.
- constraints which represent unification problems on sets, for instance $x \cup y = \{1, 2\}$. These constraints cannot produce bindings for x and y, since set-unification is currently not employed in our model. Instead, they must residuate until other constraints provide bindings.

3 The User Interface

ZAP is embedded in the diverse interfaces (graphical, XEmacs, batch) provided by ZETA [6]. Here, we look at the batch-oriented (command line) interface. Similar principles apply to the other interfaces.

3.1 Queries

Execution is involved by the two single-letter commands `e` (execute expression) and `p` (execute predicate). The output of the `help` command gives the following syntax for these commands:

```
e [-u <UNITNAME>] [ ( -r | -t | -f ) ] ( -n | -d | -o | <EXPR> )
p [-u <UNITNAME>] <PREDICATE>
```

The optional unit name specifies the section focus, as usual with ZETA commands. The first group of options to the `e` command has the following meaning:

- `-r` – "raw mode" – indicates that sets in the result should not be enumerated.
- `-t` – "try mode" – indicates that sets in the result should be enumerated to a certain extend. This may fail, producing "unresolved constraints" diagnostics. This is the default.
- `-f` – "force mode" – indicates that sets in the result should be forced to be completely enumerated. This may lead to non-termination.

[5] It is planned to integrate resolution techniques for arithmetic constraints in future versions of ZAP.
[6] zeta.html

The last group of options of the `e` command allows us to step through the solutions to the entire specification or a set-result. Consider the (loose) specification:

section *Stepping*

$$S : \mathbb{P}\,\mathbb{Z}$$
$$S \in \{\{1,2\},\{3,4\}\}$$

Evaluating S yields in:

S

```
⇛ {1,2}
   (possibly more solutions)
```

Typing `e -n` shows the next solution to S, `{3,4}`. From the current value we are observing, we can also step downwards (if the current value is a set value): typing `e -d` now yields `3 (possibly more solutions)`, and `e -n` afterwards `4`. With `e -o` we return to the outer target.

3.2 Profiling

ZAP provides two options for timing and profiling. First, with the command `xconf ton`/`xconf toff` timing of execution can beactivated and deactivated.

Second, the command `xprof` can be used to display a profile which counts the number of steps executed in each individual constraint.

4 Controlling Search

The execution model of ZAP uses concurrent constraint resolution, employing the so-called *Andorra Principle* to schedule search. This means that deterministic computation is preferred for non-deterministic computation: on resolving the constraints of a goal, those constraints which try to create a choice point are suspended until no other constraints can continue.

If several choices remain in such a situation, those which are textual "first" are preferred. However, for recursive definitions, also those choices which are "older" are preferred for those which are "younger". This is necessary in order to provide fairness. It may lead to unexpected behavior, which is discussed in this section.

Consider a specification of the 8-queens problem:

section *queens*

$$N == 8$$

$$Pos == \mathbb{Z} \times \mathbb{Z}$$

$$col == \lambda\, p : Pos \bullet p.1$$
$$row == \lambda\, p : Pos \bullet p.2$$

$$up == \lambda\, p : Pos \bullet col(p) - row(p)$$
$$down == \lambda\, p : Pos \bullet col(p) + row(p)$$

$$feasible == \{ps : \mathbb{P}\, Pos;\ p : Pos \mid \forall\, p' : Pos \mid p' \in ps \bullet col(p) \neq col(p') \land row(p) \neq row(p') \land up(p) \neq up(p') \land down(p) \neq down(p')\}$$

$$queens : \mathbb{N} \leftrightarrow \mathbb{P}\, Pos$$

$$\begin{aligned} queens = {} & \{(0, \varnothing)\} \cup \\ & \{n, c : \mathbb{N};\ qs, qs' : \mathbb{P}\, Pos \mid \\ & \quad n > 0;\ (n-1, qs) \in queens;\ c \in 1 \,..\, N \\ & \quad (qs, (c, n)) \in feasible;\ qs' = qs \cup \{(c, n)\} \bullet (n, qs')\} \end{aligned}$$

Execution of the goal

$\{qs : \mathbb{P}\ Pos \mid (8, qs) \in queens\}$

leads to virtual endless search. What is wrong with the specification (resp. ZAP's current execution model)?

The problem is that *queens* is computed again and again for the same n. Though on the same recursion level the constraint $(n-1, qs) \in queens$ preceed the constraint $c \in 1\,..\,N$ and is thus priorized, $(n-2, qs) \in queens$ is not priorized for $c \in 1\,..\,N$ on the upper level, since older constraints are priorized over younger ones. Thus for each choice point of $c \in 1\,..\,N$ the constraint $(n-2, qs) \in queens$ is backtracked and recomputed.

In fact, there is no actual dependency between $(n-1, qs) \in queens$ and $c \in 1\,..\,N$, and thus backtrack is not necessary. However, ZAP is currently not capable of recognizing the independency.

There are several ways how a user can control search, discussed below.

4.1 Sequentialization

There are two primitive predicates, seq+ (\BEGINSEQ) and seq− (\ENDSEQ) which can be used to enforce sequential execution. Used for queens, this looks as follows:

$$
\begin{array}{|l}
queens_1 : \mathbb{N} \leftrightarrow \mathbb{P}\ Pos \\
\hline
queens_1 = \\
\quad \{(0, \varnothing)\} \cup \\
\quad \{n, c : \mathbb{N};\ qs, qs' : \mathbb{P}\ Pos \mid \\
\quad\ n > 0;\ \mathsf{seq}{+};\ (n-1, qs) \in queens_1;\ c \in 1\,..\,N;\ \mathsf{seq}{-} \\
\quad\ (qs, (c, n)) \in feasible;\ qs' = qs \cup \{(c, n)\} \bullet (n, qs')\}
\end{array}
$$

The constraints enclosed in seq+ and seq− are strictly executed sequential. Note that this means, if one of them residuates, the entire group residuates. For the example, we thus ensure that choices for c are always appended to the bottom of the choice tree for qs.

4.2 Extensionalization

We can enforce the complete enumeration of the solutions on the next recursion level using the $\mathsf{ext} : \mathbb{P}\ A \nrightarrow \mathbb{P}\ A$ (\EXT) function from the toolkit. This functions takes a set and yields its extensional (enumerated) version:

$$
\begin{array}{|l}
queens_2 : \mathbb{N} \leftrightarrow \mathbb{P}\ Pos \\
\hline
queens_2 = \\
\quad \{(0, \varnothing)\} \cup \\
\quad \{n, c : \mathbb{N};\ qs, qs' : \mathbb{P}\ Pos \mid \\
\quad\ n > 0;\ qs \in \mathsf{ext}\{qs : \mathbb{P}\ Pos \mid (n-1, qs) \in queens_2\};\ c \in 1\,..\,N \\
\quad\ (qs, (c, n)) \in feasible;\ qs' = qs \cup \{(c, n)\} \bullet (n, qs')\}
\end{array}
$$

Here, $qs \in \{\ldots\}$ is still backtracked if $c \in 1\,..\,N$ makes a choice, but since the result of $\{\ldots\}$ is already computed, this does not hurts very much.

The ext function can enhance efficiency in many other situations which deal with finite sets. In general, if a set is defined by a set-comprehension, its contents is recomputed at every application point. This behavior is intended, since the application context determines what we actually need to compute of a set (in a constraint $e \in S$, the "pattern" of the expression e narrows what to compute from S). However, for problems such as *queens* this behavior is not adequate.

5 Special Definitions

The mathematical toolkit which comes with ZETA contains some special definitions which are tailored for working with ZAP and are documented below.

5.1 Debug Printing

The function $\mathsf{trace}\ x$ (\TRACE) can be used to print the value of x to the session log. It just returns x. The function $\mathsf{print}(x, y)$ (\PRINT) prints the value of x and returns y. The semantic definition, apart of these side-effects, is as follows:

$$[X] \qquad \mathsf{trace} == \lambda\, x : X \bullet x$$

$$[X, Y] \qquad \mathsf{print} == \lambda\, x : X;\ y : Y \bullet y$$

5.2 Forcing

The function $\mathsf{force}\ x$ (\FORCE) residuates the executing constraint until x is bound (if x is not a variable, it immediately returns). The function $\mathsf{deepforce}\ x$ (\DEEPFORCE) does so until *all* free variables in x are bound. Semantically, both are identity:

$$[X] \qquad \mathsf{force} == \lambda\, x : X \bullet x$$

$$[X] \qquad \mathsf{deepforce} == \lambda\, x : X \bullet x$$

5.3 Assumptions

The generic constant $\mathsf{assumed}\ X$ (\assumed) takes a set and converts it into a unversal set. Semantically, it is identity:

generic $1(\mathsf{assumed}\ _)$

$$[X] \qquad \mathsf{assumed}\ _ == X$$

However, the compiler treats $\mathsf{assumed}\ X$ as $\{x : X\}$, discarding any constraints involved by the instance of X.

This allows a usage as follows:

$$S$$
$$x : \mathbb{Z}$$
$$\mathsf{assumed}[|\ x > 0]$$

The constraint $x > 0$ is not checked at execution time.

There is also a ZETA macro which supports the above usage in a more natural notation:

```
%%macro \assert 1 [|#1]
```

$$S$$
$$x : \mathbb{Z}$$
$$\textbf{assert}\ x > 0$$

5.4 Extensionalization

The function ext x (\EXT) enforces the enumeration of the set x, denotating its extensional internal representation. It is undefined for sets which cannot be enumerated. The semantic definition is identity:

function 90(ext _)

$$[X]$$
$$\mathsf{ext}\ _ == \lambda\, S : \mathbb{P}\, X \bullet S$$

5.5 Reduction Over Sets

The function *setreduce* implements homomorphisms over sets:

$$[X, Y]$$
$$setreduce : (X \times Y \pfun Y) \times Y \times \mathbb{P}\, X \pfun Y$$
$$setreduce = \lambda\, f : X \times Y \pfun Y;\ y : Y;\ S : \mathbb{P}\, X \bullet$$
$$\quad \mathbf{if}\ S = \varnothing\ \mathbf{then}\ y\ \mathbf{else}(\mu\, y' : Y \mid \forall\, x : X \bullet y' = setreduce(f, f(x, y), S \setminus \{x\}))$$

Semantically, it is thus only defined for functions f which are commutative in the first argument $(f(x, f(x', y)) = f(x', f(x, y))$. However, this condition is not checked in the actual implementation.

5.6 Reduction Over Sequences

Folding on sequences, as well-known from functional programming, is provided:

$$[X, Y]$$
$$foldr : (X \times Y \pfun Y) \times Y \fun \mathrm{seq}\, X \pfun Y$$
$$foldr = \lambda\, f : X \times Y \pfun Y;\ y : Y \bullet \lambda\, s : \mathrm{seq}\, X \bullet \mathbf{if}\ s = \langle\rangle\ \mathbf{then}\ y\ \mathbf{else}\, f(head\ s, foldr(f, y)(tail\ s))$$

$$[X, Y]$$
$$foldl : Y \times (Y \times X \pfun Y) \fun \mathrm{seq}\, X \pfun Y$$
$$foldl = \lambda\, y : Y;\ f : Y \times X \pfun Y \bullet \lambda\, s : \mathrm{seq}\, X \bullet \mathbf{if}\ s = \langle\rangle\ \mathbf{then}\ y\ \mathbf{else}\, foldl(f(y, head\ s), f)(tail\ s)$$

5.7 Extensional Powerset

The function $\mathbb{F}_{\mathsf{E}}\, X$ (\efinset) constructs the extensional powerset of the set X, which must be enumerable. Semantically, it is the same as $\mathbb{F}\, X$:

function 90($\mathbb{F}_{\mathsf{E}}$ _)

$$[X]$$
$$\mathbb{F}_{\mathsf{E}}\ _ == \lambda\, S : \mathbb{P}\, X \bullet \mathbb{F}\, X$$

We do not define $\mathbb{F}$ itself to deliver an extension since for even small domains powersets easily explode – $\mathbb{F}_{\mathsf{E}}(1 \mathinner{..} 16)$ contains 65536 elements.

5.8 Reflection

A simple concept of reflection is provided, which is currently only used for type-safe conversion of data given in a denotation or a file into a runtime value (next section).

The function typeinfo T results a value representing information of the base type of the set T. The function giventypeinfo T results information about a free type definition associated with the

base type of T, which must be a simple type name. These functions can only be used if T does not contains generic types.

$[TYPEINFO[A]]$ $[GIVENTYPEINFO[A]]$ **generic** 10(typeinfo _)
generic 10(giventypeinfo _)

$[A]$
typeinfo _ : $TYPEINFO[A]$

$[A]$
giventypeinfo _ : $GIVENTYPEINFO[A]$

5.9 Data Input

For the purpose of e.g. test evaluation, ZAP provides a type-safe conversion of data found in a file or in a denotation into values. This is achieved by the following functions:

function 9(_ fromfile _)
function 9(_ fromdeno _)

$[A]$
_ fromfile _ : $TYPEINFO[A] \times \mathsf{denotation} \pfun \mathrm{seq}\, A$

$[A]$
_ fromdeno _ : $TYPEINFO[A] \times \mathsf{denotation} \pfun \mathrm{seq}\, A$

These functions parse the denotation or the contents of the file according to the following grammar, parameterized over types, with P_τ the start symbol, reading a sequence of values, and V_τ the non-terminal reading a single value:

$$
\begin{array}{ll}
P_\tau & \to \epsilon \mid P_\tau\ V_\tau \\
V_{\mathbb{Z}} & \to \texttt{number} \\
V_{\mathbb{P}\,\tau} & \to \{\, V_\tau \ldots V_\tau \} \\
V_{\tau_1 \times \ldots \times \tau_n} & \to V_{\tau_1} \ldots V_{\tau_n} \\
V_{[x_1:\tau_1;\ \ldots;\ x_n:\tau_n]} & \to V_{\tau_1} \ldots V_{\tau_n} \\
V_{T ::= cons_1 \langle\!\langle \tau_1 \rangle\!\rangle \mid \ldots \mid cons_n \langle\!\langle \tau_n \rangle\!\rangle} & \to \texttt{cons}_1\, V_{\tau_1} \mid \ldots \mid \texttt{cons}_n\, V_{\tau_n}
\end{array}
$$

Tokens are numbers, constructor names, and braces. Arbitrary white space is allowed between tokens. Here is an example:

section *DataInput* $Seq[A] ::= nil \mid cons \langle\!\langle A \times Seq[A] \rangle\!\rangle$

typeinfo $Seq[\mathbb{P}[x : \mathbb{Z};\ y : \mathbb{Z}]]$ fromdeno "`cons {1 2} cons {} cons {5 6 7 8} nil nil`"

$\Rrightarrow$ `<cons({<x==1,y==2>},cons({},cons({<x==5,y==6>,<x==7,y==8>},nil))),nil>`

References

[1] Wolfgang Grieskamp. *A Set-Based Calculus and its Implementation.* PhD thesis, Technische Universität Berlin, 1999.

[2] Wolfgang Grieskamp. A Computation Model for Z based on Concurrent Constraint Resolution. To appear in ZB2000 – International Conference of Z and B Users, September 2000.

[3] Wolfgang Grieskamp and Markus Lepper. Encoding Temporal Logics in Executable Z: A Case Study for the ZETA System. In *LPAR 2000*, November 2000. to appear.

[4] Wolfgang Grieskamp and Markus Lepper. Using Use Cases in Executable Z. In *ICFEM 2000*, September 2000. to appear.